Memglide: Measuring the Cost and Latency of Bounded Repository Context for Coding Agents

SREE CHARAN REDDY KAILASAM

Providing repository context before a coding task can reduce search work, but preparation and verification also consume time. Memglide selects bounded current-source context using lexical and static syntax evidence and can supply verified public commands even when source selection abstains. We compare the installed workflow with pass-through Codex on an amended fixed panel of 28 author-designed feature tasks, with two repetitions per method: 112 native sessions. Both methods pass 54 of 56 frozen strict assessments. At fixed prices applied to measured tokens, mean API-equivalent cost is $0.16094 for Codex and $0.15316 with Memglide, an observed 4.84% reduction. Mean full-workflow time is 59.59 and 60.29 seconds, respectively, an observed 1.17% increase. Paired task-level 95% intervals for Memglide minus Codex are [-$0.02524, $0.00968] and [-3.02, 4.42] seconds; neither resource-superiority test passes. Native execution is 2.06 seconds shorter on average, offset by 2.75 seconds of added workflow overhead. All 128 sessions in the completed comparison pool have valid measurement and grading records; 16 previously exposed task sessions remain descriptive. Source-policy and transport-handling amendments followed partial collection, so inference is nominal rather than a fully prospective confirmation. The study identifies an overhead bottleneck and preserves the uncertainty: token-price estimates are not invoices, equal observed success counts do not establish quality equivalence, and joint cost-and-speed superiority is not established.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Information systems** → *Information retrieval*.

Additional Key Words and Phrases: coding agents, repository context, source retrieval, reproducibility, software engineering

1 Introduction

Coding agents must find the source and test commands needed for a change. Providing that information in advance may reduce their work, but preparation also costs time and adds to the prompt. The relevant question is whether the complete workflow improves after selection, verification, prompt length, failed attempts and preservation of existing behavior are counted. A compact source bundle or shorter serialized response cannot settle that question.

Memglide is a local repository-context system with a native coding-agent launcher. The launcher uses lexical and static syntax evidence to select current source and delivers complete selected files within fixed limits. Its opt-in `commands-first` workflow can supply verified public test commands and observed baseline counts even when source selection abstains. Source selection and command assistance therefore have separate activation conditions: an ambiguous source target need not make current, verified commands unusable. Preparation uses no auxiliary language model or embedding service. The external coding model interprets the task, edits the code and runs checks.

Earlier experiments revealed gaps between a working context system and an efficiency claim. Interactive MCP retrieval could increase tokens and elapsed time. An automatic selector could abstain on most tasks, producing apparent agent differences without evidence of a preload effect. A development-only compiler configuration could change a score in a separate compatibility diagnostic. A provider failure could leave token usage unknown. These findings motivated a comparison of the installed system with explicit records of activation, source protection, timing and missing data.

Development included a 30-round search over prompt and verification approaches on three tasks and subsequent native comparisons. We retain those experiments as exploratory evidence. The latest comparison evaluates pass-through Codex against the installed `commands-first` candidate, with demand-driven public verification. The primary cohort contains 28 tasks, each run twice per condition, for 112 sessions. A further 16 sessions on previously exposed tasks remain

Author's Contact Information: Sree Charan Reddy Kailasam, srcharan014@gmail.com.

in the 128-session amended analysis. Private grading occurs after native collection closes; reference implementations, oracles and revealing plaintext logs are sequestered during collection.

The tasks are feature additions to small public libraries, designed with AI assistance. SWE-bench, by comparison, evaluates patches for real GitHub issues [2]. The primary cohort was fixed before its native sessions, but collection required subsequent corrections to evaluation infrastructure. These included identifying published production files that an edit-protection rule had misclassified, and distinguishing completed-turn transport retries from fatal failures. Original records, failures and amendments remain in the evidence. The analysis is an amended fixed-panel comparison with nominal inference, rather than an untouched prospective confirmation. Its purposive task selection also limits generalization to other coding workloads.

On the 28-task primary cohort, Memglide's mean uniform API-price equivalent was 4.84% lower, while mean measured workflow time was 1.17% higher. Both conditions passed the strict finite correctness checks in 54 of 56 sessions. The resource intervals include zero, and the specified joint cost-and-time superiority criterion was not met. The lower tariff point estimate does not establish reliable savings; the timing result does not support a speedup. The tariff is computed from native-reported tokens under a fixed rate schedule, not from provider invoices.

We ask three questions. First, when does the installed policy deliver source, verified commands, both or neither? Second, how do the conditions compare under finite feature, original public-check and source-integrity requirements? Third, how do native token-tariff equivalents and workflow time change when fresh indexing and public baseline verification are included?

The contributions are an implementation that separates bounded source selection from verified command assistance; an evaluation method that records activation, functional outcomes, timing and telemetry provenance; and a development-to- evaluation record that retains unsuccessful approaches and collection repairs. The study evaluates context and workflow preparation around an existing model. It does not train model weights, measure memory-reuse benefits or establish a benchmark-leading coding agent.

2 Related work

Repository context has been studied as both a retrieval problem and part of code generation. RepoCoder combines a similarity retriever with a pretrained code model in an iterative retrieval–generation pipeline [7]. RepoBench evaluates retrieval, completion and a combined pipeline, including cross-file dependencies [4]. These settings separate the evidence a retriever returns from the result of using it. Memglide evaluates functional patches with preserved public regressions on small feature tasks, rather than code-completion similarity.

Context length alone gives an incomplete account of useful input. Liu et al. varied the position of relevant information in multi-document question answering and key–value retrieval. Performance often declined when relevant information appeared in the middle of long contexts [3]. LLMLingua compresses prompts through a coarse-to-fine method with token-level compression and model alignment [1]. Memglide uses lexical evidence and static syntax analysis to construct bounded bundles of selected complete files. Preparation makes no auxiliary model call and can abstain when selection or freshness checks fail. The present study includes no performance comparison with these approaches.

Agent design provides a second point of reference. SWE-agent studies an agent–computer interface for automated software engineering [6]. Agentless separates localization, repair and patch validation [5]. Memglide runs before the installed coding agent, supplying source and optional verified-command context while leaving investigation and editing to the agent. SWE-bench evaluates issue-to-patch behavior on real GitHub issues [2]. The present panel uses

author-designed extensions to public repositories. Its results describe an installed-system comparison within that panel; they supply neither a SWE-bench score nor a direct performance comparison with these systems.

3 Architecture and evidence boundaries

Memglide prepares repository context and records the source and policy used to prepare it. Its central design choice is conservative selection from current local files. Lexical evidence, static syntax and bounded dependency expansion produce a small source bundle without an embedding model or auxiliary language-model request. The coding agent interprets the task, edits the repository and checks its work. Memglide changes context preparation rather than model weights.

The implementation has two entry paths. Interactive clients reach a common service through a command-line interface, loopback HTTP API or MCP STDIO server. The service combines repository indexing, context retrieval, a SQLite database and explicit session memory; a dashboard uses the HTTP API. The native launcher, `memglide-codex`, calls the source-preparation libraries directly before executing Codex. The latest comparison evaluates this path with the opt-in `commands-first` candidate at commit `3c3f82b`, together with the measured setup planner described below. Figure 1 distinguishes this path from the interactive services. Interactive retrieval, the dashboard and session-memory reuse require separate evaluation.

Repository registration resolves the Git root and records project state. The SQLite FTS5 index stores searchable source, file hashes and metadata, symbol locations and signatures, and conservative dependency edges. Python structure comes from the standard AST; JavaScript and TypeScript use an isolated Tree-sitter worker. Incremental refresh checks current files before using indexed evidence. Dynamic imports, generated code and runtime dispatch can leave static dependencies unresolved, limiting what the selected context represents.

The interactive context engine combines identifier and path matches, normalized terms, FTS5 evidence and graph expansion. Session memory stores caller-submitted observations, decisions, errors and fixes with provenance, timestamps and source versions. Sessions have inactivity and absolute expiry limits. Returned context passports identify source hashes, memory versions and omissions. These features provide versioned context within an active session; the experiment does not measure cross-session reuse. The native prelaunch selector follows a separate deterministic algorithm and does not automatically inject session observations.

Prelaunch selection first checks whether preparation is enabled, context has already been supplied, and an approved project and eligible task are available. Explicit off/context-supplied modes and requests failing these initial approval or task gates preserve the original task. The v4 policy then securely refreshes the index and screens production files. Known tests, examples and demonstration directories are excluded from production competition. Potentially relevant unsupported files, partial indexing or an overfull candidate frontier cause source-selection abstention. Under `commands-first`, a completed content-selection abstention may still be eligible for verified command assistance. Source activation and command activation are recorded separately; reading source successfully does not establish that the task received useful context.

Remaining supported candidates are parsed into runtime or declaration syntax units. Normalized query terms receive candidate-set frequency weights $w(t) = \log(1 + (N + 0.5)/(\mathrm{df}(t) + 0.5))$. Here N is the number of files passed to syntax ranking, and $\mathrm{df}(t)$ counts those files with term t in the clean body or header of any eligible-role syntax unit. A file counts once even if several units match; this frequency is computed on the bounded candidate set, rather than the entire repository. A unit is sufficient if it has an explicit current anchor, or at least two distinct clean body/header matches including one body match. Explicit and sufficient evidence receive large priority terms in ranking. The remaining score

combines body, header, path and documentation weights with coefficients 3, 2, 0.25 and 0.05. This score orders evidence; it is not a calibrated confidence. An independently competitive file causes abstention unless the leader has the required body-evidence dominance.

After accepting an anchor, Memglide reads the complete file and at most three resolved direct production helpers. It checks their current hashes and selection evidence before rendering the task. “Full source” refers to these selected files, not repository-wide or transitive dependency coverage. Final handoff revalidates inspected competitors, selected source, approval and ignore policy. A changed proof prevents reuse of the prepared context. Source, comments and strings remain project data for the coding agent to interpret.

Each phase has its own bounds. Default indexing permits 20,000 files and 100 MiB of eligible source, with a 512,000-byte per-file read limit. The v4 task limit is 4,000 UTF-8 bytes and 64 normalized non-generic terms, each at most 128 characters. Coarse screening permits 100 files and 2 MiB; syntax analysis permits 24 files and a separate 2 MiB. The syntax worker has a five-second timeout, 200,000-node limit and 1,000,000-byte output limit. Added context defaults to 12,000 UTF-8 bytes, with an additional `ceil(bytes/3)` planning limit of 4,000. This proxy is not a provider token measurement. Repeated freshness checks mean that phase byte limits are not whole-workflow I/O or memory bounds.

Public-command assistance requires explicit verification. The caller declares atomic argument lists and allowed environment updates. A verifier records a fresh bounded public-text tree, fingerprints declared executable files, runs the checks, and records exit status, positive behavioral-test counts and log hashes. It checks the tree, policy and tools again before writing a canonical receipt outside the project. The caller retains the trusted receipt hash. Because public tests may execute repository code, verification is a separate authorized action; context preparation does not itself run those commands. Public-tree binding is limited to 20,000 directory entries, depth 12 and 16 MiB of eligible text; individual reads also obey the project’s source-size limit. A receipt is at most 4 MiB, with at most 16 declared commands and bounded arguments. Declared executable fingerprints use a separate 256 MiB per-tool limit. These limits describe different inputs and are not a combined workflow resource budget.

The evaluated candidate separates source selection from command delivery. Its three final modes are:

- `preloaded`: complete selected source is delivered; a current verified command card may accompany it.
- `commands_only`: a card is delivered without selected source. The receipt retains the source-decline reason and selection proof, and its source list is empty.
- `passthrough`: neither source nor a command card is delivered, and the original task bytes are preserved.

Command-only eligibility is limited to explicit content-selection declines, including ambiguity, weak or missing production evidence, an unsupported competitor and bounded candidate, dependency or context limits. It requires a completed v4 decision with a refreshed index and no selected files. Explicit caller skip gates, an over-limit query, missing approval and freshness or policy failures do not authorize a command card. This exception changes which verified facts can be delivered; it does not relax the v4 source-selection threshold or substitute guessed source.

The launcher loads the receipt against the caller’s trusted hash and rechecks its public-tree, approval, policy and tool bindings. The card gives atomic commands, allowed environment settings and observed baseline results. Its `commands-first` instruction says that the listed checks passed on the current starting tree, asks the agent to use that baseline without repeating it before editing, and requires the listed checks after changes with their full results. This is an instruction backed by observed baseline facts, not evidence that the agent obeyed it or that the future implementation passes.

Unavailable or stale optional facts, or a card that exceeds the unchanged whole-context caps, leave the underlying source preparation available. For a command-only candidate, this means returning to the original task when no card can be delivered. Invalid explicitly pinned receipt input is rejected on an active workflow path. Before native execution, the launcher repeats source selection, validates the complete source or source-decline proof, reconstructs the rendered payload exactly, checks its hashes and rechecks the provider's current bindings. Even an empty source list does not exempt the eligible public tree or executable tools from this final freshness check.

The evaluated setup orchestration adds a demand-driven planner before optional provider creation. After indexing, it invokes the installed v4 preparation and validation functions, then applies the same command-only eligibility predicate. If neither complete source nor command-only assistance is eligible, setup skips public-command execution and supplies no provider to the ordinary launcher. Eligible routes still require the full fresh binding, public checks, cleanup and current-manifest validation. A typed initial binding unavailability can omit the optional provider; failed checks and later integrity failures remain errors. The launcher subsequently repeats routing and final validation. The planner's observation is not a reusable receipt. This optimization belongs to the measured setup workflow; the shipping launcher still consumes an explicitly verified provider rather than silently running repository tests. Planning, indexing and verification time are included in the workflow measurement.

The trust boundary separates local preparation from an external coding model. Selected source is sent to Codex; local-first describes where the index and state reside. Secure readers enforce containment, reject symlink traversal and read bounded regular UTF-8 files. Known sensitive filenames and content patterns are filtered heuristically. Hashes establish equality at checked reads, not semantic safety, permanent freshness or universal secret detection. A command receipt covers eligible public text and declared executable bytes, not every transitive dependency, binary resource or undeclared environment setting. Sensitive or unreadable eligible text can make the provider unavailable; the system then makes no verified-command claim. Command-runner time and output limits also differ from operating-system isolation.

These boundaries motivate separate evaluation of activation, functional success, preparation time, native execution time and token-tariff equivalents. The source-and-command workflow incurs planning, verification and freshness costs. Finite public and hidden checks do not establish universal correctness, semantic quality of new tests or general coding superiority. Natural abstentions, functional failures and unavailable telemetry remain in the record. The measured native path provides no evidence of benefits for the implemented memory, interactive API or dashboard features.

4 Evaluation design

4.1 Task panel and conditions

The v7 experiment started with 36 author-designed feature tasks in 36 pinned public repositories: 18 Python projects and 18 JavaScript/TypeScript projects. Tasks were selected for executable upstream checks, not sampled from historical issues. Several repositories share maintainers or development conventions. The model's prior familiarity with their source is unknown. AI agents assisted with task writing, feature oracles, reference implementations, deliberately incorrect controls and review. Separate authoring roles and control outcomes provide checks on the evaluation; they are not independent human replication. Public task inputs, dependency profiles, commands and original protected-file hashes were fixed before collection. Each task had baseline, reference and incorrect-implementation controls. Private tests remain finite specifications, and the wording ambiguity discussed in the results limits one task's rubric.

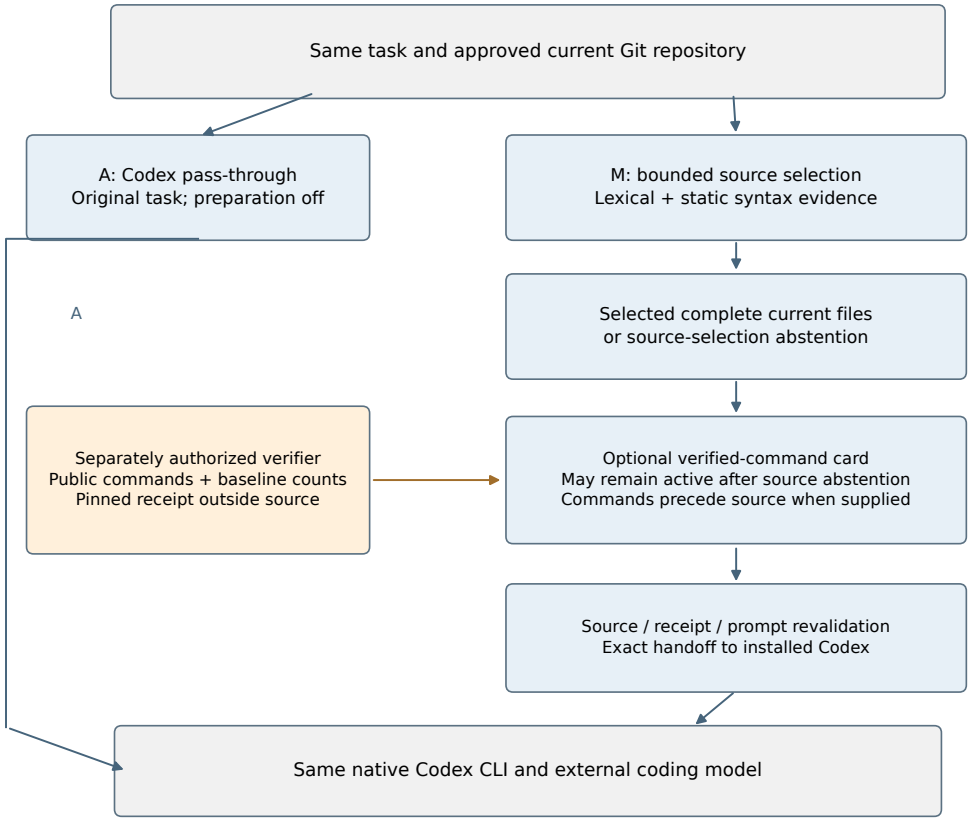


Fig. 1. The evaluated A/M workflow. A disables preparation. M uses bounded source selection and opt-in verified commands, including a commands-only route after source abstention. The separately authorized verifier produces baseline facts; it is not an automatic model call. Interactive memory and dashboard services are outside the measured path.

The comparison uses the installed candidate whose source bytes are preserved at commit `3c3f82b`. Arm A invokes its explicit off/pass-through mode. Arm M uses the commands-first policy: bounded v4 source discovery can preload source; when source discovery abstains but the production workflow is eligible, a verified public-command card can be supplied without source. Otherwise the original task passes through. All assigned tasks remain in the comparison, including abstentions. Source, command-only and pass-through decisions are policy outcomes, not separate randomized arms.

M indexes the project and checks actual production eligibility before creating the command provider. Full public baseline verification and binding are skipped only when no eligible card would attach. Eligible source or command-only paths still require fresh verification, cleanup, manifest binding and validation by the installed launcher. This demand-driven setup policy was fixed before the 28-task primary cohort began. Product source did not change during that cohort; later repairs concerned the harness, source-role metadata and telemetry interpretation. The experiment measures the combined installed policy, not the isolated effect of a source excerpt or command card.

4.2 Allocation and the fixed primary cohort

The original schedule contained two arms and two repeats per task, or 144 unique assignments. Seed 20261101 determined a sequential, balanced first-repeat schedule with 18 A–M and 18 M–A task orders. Repeat two reversed the full first-repeat task/arm sequence. Every coding session used a fresh project copy, local native state and thread.

The first 15 assignments exposed eight tasks before an infrastructure stop. Assignment 0016, on an already exposed task, then failed locally before the native bridge because its copied executable lacked execute permission. Its recorded usage remains unknown. Before any of the remaining tasks had a model session, the primary cohort was defined as the 28 tasks outside that exposure prefix: 15 Python and 13 JavaScript/TypeScript projects. Each contributes both arms twice, giving 112 sessions and 28 analysis clusters. This membership rule used exposure, not observed savings, speed or grading outcomes.

The same 112 primary assignments occupy slots 0017–0128. The final 16 slots are the remaining second-repeat sessions on the eight exposed tasks; these are auxiliary descriptive evidence. The completed 128-record comparison pool therefore spans 36 tasks with unequal repeat coverage. Statistical inference uses the 28 complete task clusters; the other 16 records are descriptive. No primary task was removed because retrieval abstained, coding failed or a measurement appeared unfavorable. The primary cohort retains 13 A–M and 15 M–A first-repeat orders.

4.3 Collection, amendments and isolation

All coding sessions requested `gpt-6.1-sol`, low reasoning effort and the default service tier through Codex CLI 0.159.2. This fixes the requested alias, not a provider backend or model-weight snapshot. Both arms received the same task and common developer contract, response schema, tools and network-disabled workspace-write configuration. Project-document injection was disabled in both. Actual prompt bytes and native arguments were checked against installed zero-model handoff receipts. These are coding-agent `codex exec` sessions; local sandbox processes used for controls or grading do not themselves make additional coding-model requests. Neither process counts nor agent commands identify the provider's internal inference count.

Collection required several versioned amendments. After 25 primary measurements, a broad protected-file rule incorrectly classified published production files as tests or build material. A baseline-only audit of all 36 repositories certified five exact production code/type paths. It used original package metadata and public module documentation, not candidate patches, private grades or resource contrasts. The correction was applied uniformly to all 128 records while preserving the original raw flags. Feature assertions and reference cases were unchanged. Ordinary edits to owned protected tests, documentation or configuration still imply strict coding failure; unsafe paths, Git/shared-dependency changes and evidence corruption remain hard integrity failures.

After 52 measurements, the usage parser stopped on an intermediate reconnect warning even though its turn completed. The pinned CLI source distinguishes retry notifications from fatal failures but renders both through the same JSONL error shape. A separate, source-verified adapter accepts only the narrow retry constructions within one fresh, completed thread/turn, with matching full token counters, telemetry, process identity and final-output evidence. It recomputes every pooled record and preserves the original parser verdict and events. Arbitrary errors or unfinished turns remain invalid. The resulting estimand is native-reported completed-thread usage, not proof that every underlying provider attempt is accounted for in an invoice.

A subsequent slot-0069 CLI startup failed on a missing local output-schema file. Pinned source established that this exit precedes model-turn creation. That extra process remains a separate operational failure with its original null usage;

only its separate ledger records source-proven zero generation requests and zero model-generation tariff. The canonical model assignment proceeded after deployment repair. A declared sensitivity charges the full failed startup duration to that M assignment. Operator diagnosis and repair time was not measured. Overall accounting retains 144 unique master assignments, 145 allocation attempts, 143 coding-model turns and 144 native CLI processes. These counts exclude separate development and connectivity probes.

The final pool combines the first 52 measurements with the remaining 76 canonical sessions. There were no retries of measured coding trials, replacement tasks, additional outcome-driven repeats or significance-based stopping. Original stopped states and protocol decisions remain unchanged and cannot acquire a retrospective confirmatory pass. Because 25 measurements preceded the source-role correction and 52 preceded the parser amendment, the final study is an **amended fixed-panel evaluation**. Its local hashes and analysis freezes are audit commitments, not a public preregistration or a fully prospective confirmation.

Private grader, reference and control plaintext was absent during model collection. Authenticated AES-GCM ciphertext and per-file commitments could remain locally, while keys were held outside the model executor. A historical key-loss incident required reconstruction against original hash commitments before the primary cohort began; later versioning changed source-protection metadata and deployment paths, not feature assertions. The final private bundle was restored and byte-verified after collection closed. This is a documented plaintext-sequestration procedure, not proof of complete operating-system isolation.

4.4 Timing and strict assessment

Timing begins after identical public-source copying and Git-baseline creation, before registration, indexing, eligibility planning or verification. It ends after native process-group cleanup. All M preparation, fresh eligible public checks, generated-file cleanup and installed-launcher work are included. Grading, environment acquisition, source/Git copying and evidence packaging are excluded. Setup has a 90-second allowance; native execution has a separate 600-second allowance from the recorded bridge handoff. Native wall time is a local process measurement, not isolated remote inference latency.

Environment profiles and developer instructions were shared. Projects and run-local caches were fresh; OS page caches, provider prefix caches and backend load were uncontrolled. Allocation order was retained across amendments, but interruptions and clock-time effects cannot be removed by reversing order.

After closure, each final source tree was reconstructed without reusing model-generated caches. Public and private checks ran with candidate source read-only, network disabled and bounded artifact write roots. Raw hashes, source-role adjudication, native adjudication and version 3 grading metadata are bound to every result. Strict success requires valid execution and usage, correct treatment, intact hard protections, valid unchanged-source grading and passing original public plus feature checks. A valid public coding failure can conclude strict failure without running private tests. Failures retain their observed cost and time; unknown values are never filled with zero. All 128 pooled records must satisfy integrity and grading prerequisites before the primary inference is eligible.

4.5 Accounting and analysis

For each task and arm, the two repeats are averaged first. The analysis uses the 28 paired task differences M minus A for two primary endpoints: uniform API-equivalent dollars and full workflow seconds. The dollar convention is [(input minus cached input) × 5 + cached input × 0.20 + output × 15] / 1,000,000. These are native-reported cumulative token counters under a fixed historical tariff, not current prices, subscription charges or actual billing.

Each endpoint receives a paired Student t test with 27 degrees of freedom, one-sided alternative M minus A below zero and alpha 0.025, together with a pointwise 95% two-sided t interval. The joint efficiency claim requires both tests to pass, both upper interval endpoints below zero, complete primary metrics, all 128 integrity/grade gates and M's strict-success count at least A's among 56 sessions per arm. This intersection-union rule tests a single claim of simultaneous cost and speed superiority; one favorable endpoint cannot replace the other. Equal observed pass counts are a quality-count guard, not a noninferiority or equivalence test.

Intervals and tests are nominal model-based summaries. Their finite-sample calibration assumes independent, normally distributed task differences; repeats do not increase the independent cluster count above 28. Pointwise 10,000-resample task-bootstrap intervals, distribution/influence diagnostics, workflow components and jointly successful timing are secondary. A further secondary summary omits the entire four-session task cluster containing any certified retry warning, symmetrically across arms and repeats. It cannot change the primary decision. Soft known-tariff allowances and admission reserves controlled further dispatch, not actual account charges. The old 0016 unknown usage prevents a complete 144-assignment cost claim even though all 112 primary and all 128 pooled comparison records have complete uniform accounting. The public result package contains the portable export and reproduction instructions.

5 Development history

Memglide's earlier evaluations motivated the installed-policy comparison but are not pooled with it. Historical capsules preserve source snapshots, tasks, original outcomes and analysis rules. Some stages reanalyze existing attempts; they must not be added to model-session counts as new trials. These studies provide engineering context rather than a common representative benchmark.

Early native MCP experiments found that retrieval could increase resource use. One matched cohort used 2.751 times the input tokens and 1.465 times the Codex-plus-index time; an optimized cohort still increased input by 32.82% and time by 17.88%. Automatic source delivery also needed validation: an earlier v4 comparison added source on none of its 24 assisted assignments, so its between-arm differences could not establish a preloading benefit. Later active pilots exposed both delivery failures and the danger of treating cheap unsuccessful attempts as efficient coding.

The v5 development search evaluated 30 workflow changes on three tasks, with three reference sessions and 90 candidate sessions. It changed prompts and workflow construction, not model weights. Verified public commands and then baseline counts were promoted; the remaining 28 variants were rejected under the development rules. A separate eight-task comparison scheduled 32 attempts. One assisted provider failure had unknown usage and was not replaced. Its seven complete task clusters showed an 18.726% lower uniform equivalent, with a paired-bootstrap interval crossing zero. Public verification was outside that prototype's timer, and the generation framework was broader than the installed policy, limiting comparison with later studies.

The v6 installed-system study used twelve new tasks, three arms and two repeats, for 72 completed sessions. All three arms passed 24 of 24 finite strict assessments. Source-plus-command C had 13.69% lower observed uniform cost than pass-through A and 10.04% lower cost than source-only B. Neither exact cost contrast passed the two-test Holm rule; workflow timing did not establish a speed improvement. Seven tasks preloaded and five abstained. This separate exploratory result suggested that preparation overhead and abstention deserved attention. It is not the main result of this paper and is not pooled with v7.

The v7 candidate at `3c3f82b` added an opt-in verified command-only path after source abstention. A separate operational planner creates the provider only when the actual installed source/workflow checks make a card eligible.

Table 1. V7 primary panel: all 112 assigned attempts on 28 tasks. A is installed pass-through; M is the commands-first installed policy. Costs use the fixed uniform token tariff, not billing.

Arm	Strict pass	Uniform USD	Workflow (s)	Native (s)
A	54/56	0.1609423	59.5902	59.3713
M	54/56	0.1531605	60.2863	57.3127

These choices were made before the primary cohort's coding outcomes. The candidate preserves the ordinary source policy's guards; it does not force source selection when competing files make discovery abstain. The new comparison therefore measures a combined commands-first installed policy against explicit pass-through, rather than reproducing v6's three-arm component contrast.

The original v7 36-task schedule stopped after 15 assignments when source-local Vitest cache artifacts interacted with protection/capture rules. Those records remain historical evidence, and the eight tasks exposed by that prefix were excluded from the subsequently fixed primary cohort. A copied bridge's missing execute permission then stopped assignment 0016 before native execution. Its unknown usage was preserved. Source capture and cache-free deferred reconstruction were repaired before primary collection. A separate key-loss incident was resolved by recovering grader/oracle bytes against their original commitments before the primary cohort began.

Further corrections occurred after partial collection. A baseline-only publication audit corrected five exact production paths misclassified by broad protection metadata; feature assertions were unchanged. A version-pinned CLI source audit then distinguished recovered retry warnings from fatal native failures and retained completed-thread counters without editing raw events. Finally, an absent output-schema file caused a separate pre-model startup failure; source ordering established that no user generation turn began. The missing deployment inputs and preflight inventory were repaired before the remaining canonical model assignments.

These repairs retained the same 28-task primary panel, all previous primary measurements and the original assignment order. Fifty-two measured records were combined with the final 76 canonical sessions, yielding the complete 128-record pool. Raw stopped states and original decisions were never rewritten as successes. The final analysis accordingly uses nominal amended-panel inference, not a claim that the original prospective protocol completed without intervention. The v7 result package records this lineage separately from the historical studies.

6 Results

The amended v7 comparison completed all 128 canonical records with valid effective native telemetry, complete uniform accounting and valid deferred grading. The primary panel contains 112 sessions on 28 tasks, with 56 sessions per arm. Each arm passed 54 of 56 strict assessments. The full descriptive pool passed 124 of 128 assessments, or 62 of 64 per arm. Its eight previously exposed tasks contribute only their remaining second-repeat sessions; the primary inference uses the 28 complete task clusters.

6.1 Primary cost and latency

The primary uniform equivalents total USD 9.012769 for A and USD 8.5769854 for M. The observed mean difference is −0.00778185 dollars per attempt, or 4.84% lower for M. Mean workflow time is 59.5902 seconds for A and 60.2863 seconds for M: an observed increase of 0.6961 seconds, or 1.17%. These are averages over all assigned primary attempts, including the four strict-assessment failures.

Table 2. V7 paired task-mean contrasts, M minus A; 28 clusters, 27 degrees of freedom. The joint rule requires both one-sided tests at alpha 0.025 and both upper interval endpoints below zero.

Endpoint	Mean difference	Change	Pointwise 95% CI	One-sided p
Uniform USD	−0.007782	−4.84%	[−0.025242, +0.009678]	0.1843
Workflow seconds	+0.696101	+1.17%	[−3.023997, +4.416199]	0.6480

Neither endpoint meets the declared superiority rule. Both nominal intervals include zero, and joint cost-and-speed superiority is not established. The cost estimate favors M, while the workflow point estimate favors A; the data do not establish either a population cost reduction or a population slowdown. The bootstrap sensitivity agrees about uncertainty: its cost interval is [−0.023639, +0.009080] dollars and its workflow interval is [−2.796425, +4.170258] seconds. It does not replace the primary t analysis.

6.2 Coding outcomes and delivered context

All four strict failures are the two repeats in each arm on task u21 (`is-number`). Original public checks passed in all four cases. The private feature oracle expected a TypeError for four nonboolean `safeInteger` options; the submitted implementations returned false instead. The public request said "reject nonboolean safeInteger values" without specifying an exception. The frozen rubric therefore contains an ambiguity that limits interpretation of these failures. Each implementation passed 111 upstream tests and 26 of 30 private checks; only the four TypeError assertions failed. All four remain strict failures, with their measured costs and times retained. The u21 exception assertions and these four strict outcomes were not changed or rescored.

M actually preloaded source on 12 primary sessions covering six tasks, supplied command-only context on 18 sessions covering nine tasks, and passed through on 26 sessions covering thirteen tasks. A passed through on all 56. Captured handoffs matched the committed treatment receipts. This mixed policy exposure is part of the primary comparison. It does not identify a causal effect of source alone, command cards alone or active retrieval alone. Both-repeat strict success occurred on the same 27 of 28 tasks in each arm. Equal counts and zero harmful task discordances do not establish quality equivalence; the descriptive one-sided 97.5% harmful-discordance upper bound remains 12.34%.

6.3 Native time, preparation and robustness

M's observed mean native wall time is 2.0586 seconds shorter than A's. Its total workflow time nevertheless is longer. Workflow minus native time averages 2.9736 seconds for M and 0.2189 seconds for A. M's measured components include 1.4081 seconds of public verification, 0.2432 seconds of indexing and 0.8178 seconds of installed-launcher preparation per primary assignment; planning, cleanup and other local work also belong to the workflow clock. This decomposition describes where time was spent. It does not establish that any individual preparation component caused a change in agent behavior.

The recorded reconnect warning occurred in A on task u26. Omitting that whole four-session task cluster, as declared for descriptive robustness, leaves 27 tasks: observed uniform cost is 5.13% lower and workflow time 1.57% higher for M. Both nominal paired intervals still include zero. The primary retains u26. Charging the separate failed schema startup's full 2.6373 workflow seconds to canonical u08-M changes the observed workflow increase to 1.25%, with an interval of [−2.976951, +4.463340] seconds. Operator repair time was not measured. Neither sensitivity changes the

primary decision. Restricting timing to the 27 tasks passing both repeats in both arms also remains secondary and outcome-conditioned.

Primary input counters average 131,105 tokens for A and 119,320 for M; output averages 2,257 and 2,188 tokens. Cached input averages 110,091 and 99,221 tokens. These are cumulative native counters, not unique source tokens or provider computation. The alternate native-price/write scenarios remain incomplete for one primary record in each arm and are not presented as complete comparative costs. A separately implemented automated audit reproduced the complete pool, primary estimates, intervals, grading counts and accounting from retained records. It is an internal consistency check, not external replication.

The reported values come from the frozen assessment with SHA-256 fcd5ded7f3c73799cad03a1fda0554294bc974ba34625b6680ee323ef5c94b26. The portable analysis summary and result package are separate public exports with their own hashes.

7 Interpretation and limitations

The result is an observed cost reduction with uncertain magnitude and no established workflow speed benefit. It supports neither a joint efficiency claim nor a conclusion that the policies are equivalent. The intervals allow meaningful differences in either direction. Repeated sessions improve the measurement of each task but leave only 28 primary task clusters. The zero-discordance quality result still permits a substantial harmful-task probability under the descriptive bound.

The primary panel was fixed by an exposure rule before its tasks had coding sessions, but the eventual evaluation is amended rather than fully prospective. Source-role classification changed after 25 measurements and retry-warning interpretation changed after 52. A local schema deployment error also required a source-certified pre-model exception. Original stopped states, raw verdicts and measurements were retained, and the amendments did not use resource contrasts or private candidate grades to choose changes. Those safeguards limit favorable selection; they do not restore the status of the original stopped protocols or create an independent confirmation. Local hash freezes are not a public preregistration. The nominal t inference should be read with this chronology, the custom task design and its distributional assumptions.

Tasks are author-designed feature additions in small public repositories, selected for runnable checks. They are not a representative issue benchmark. Several projects share maintainers and conventions, so treating task differences as independent is an approximation. Tasks, references and graders were AI-assisted. Distinct authoring agents and negative controls do not replace independent human task review or external replication. “Unexecuted” means no earlier study coding session on that task; it does not imply source unseen by the model, a blinded retrieval algorithm or an unknown task to all engineering agents. Results cannot establish performance on large codebases, other languages, other model configurations or arbitrary historical issues.

The u21 failure pattern exposes a specific validity limit. The phrase “reject nonboolean safeInteger values” permits reasonable readings that the private oracle did not treat equally. All four implementations passed original public checks but failed the oracle’s exception requirement. Retaining the frozen failures avoids favorable rescoring, yet the resulting strict-success count cannot by itself distinguish a coding defect from that specification ambiguity. Other finite oracles may also miss defects, and passing public or declaration checks does not establish complete instruction compliance or regression-test quality.

The policy is heterogeneous: source preloading, verified command-only context and pass-through all occur in M. Its contrast with A measures their combined installed workflow. Natural activation is not randomized, and subgroup or

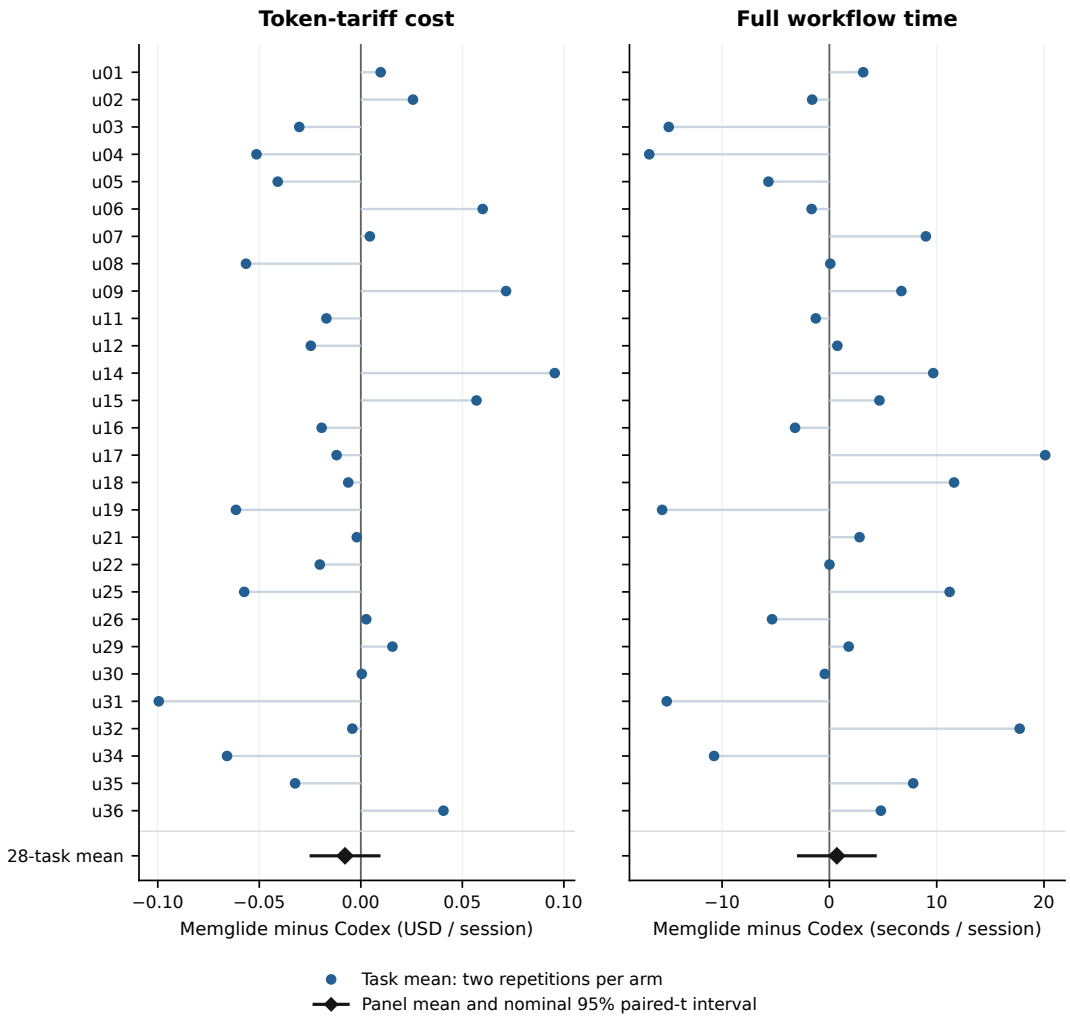


Fig. 2. Task-level Memglide-minus-Codex differences for all 28 primary tasks, averaging two repeats per method. Negative values favor Memglide. The aggregate mean and nominal 95% paired-t interval use tasks as clusters; both resource intervals cross zero.

cross-study differences cannot isolate the benefit of any component. The older v6 three-arm study used different tasks and delivery behavior; it cannot supply the missing causal ablation for v7. Demand-driven verification was implemented before the primary cohort, but its improvement over eager verification is not measured by a separate v7 arm.

Uniform dollars value native-reported completed-thread tokens at the fixed 5/0.20/15 tariff. They are not invoices, subscription charges, current model prices or proof that all unsuccessful underlying provider attempts are counted. The source-backed retry amendment establishes the meaning of the retained CLI counters, not the provider's billing behavior. Secondary price scenarios have missing records and cannot be promoted into a complete cost claim. The old 0016 usage remains unknown, preventing a complete 144-assignment accounting claim. Engineering, hosting, operator

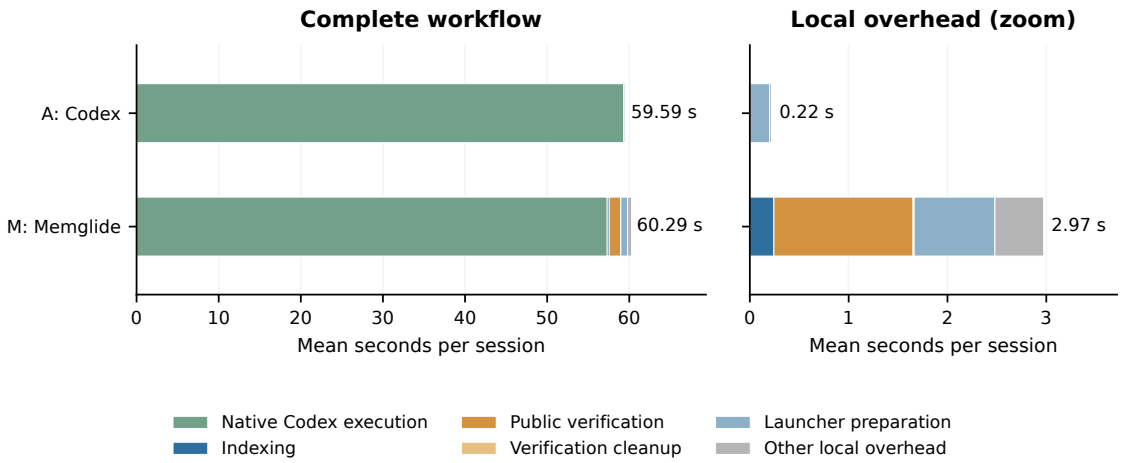


Fig. 3. Mean complete-workflow components over 56 sessions per method. Fresh indexing, public baseline verification, cleanup and launcher preparation are included. Other overhead is the workflow total less nonoverlapping recorded components. Native execution includes process handling and external-model waiting; it is not isolated inference latency.

diagnosis and repair effort are not priced. Command-text diagnostics do not measure all file I/O or avoided model inference; reported zero cache writes may reflect upstream defaulting.

Workflow latency includes index, planning, eligible verification, preparation, native execution and cleanup, but excludes source/Git copying, environment acquisition, grading and packaging. The additional failed schema startup is reported separately and charged in a sensitivity; repair minutes were not timed. Fresh project/run caches do not control OS caches, provider prefix caches, backend revisions or cloud load. Sequential execution, reversed orders and retained pauses do not remove all temporal dependence. Native wall time includes local process work and is not pure remote inference time. Amortized verification or long-lived-project operation may change the tradeoff but is unmeasured here.

Finally, bounded source and command receipts do not make model inference local or guarantee privacy. Selected text is supplied to an external model. Hashes and receipts bind declared inputs and executables rather than every dependency or environment property. Authenticated private-bundle sequestration and stronger read-only grading sandboxes are experimental controls with documented scopes, not comprehensive operating-system security proofs. Published reproduction artifacts therefore separate portable measurement data from account state, private keys and executor-specific runtime material.

8 Conclusion

Memglide provides bounded source discovery and optional verified public-command context around an installed coding agent. The amended v7 evaluation retained 28 complete task clusters and all 112 primary sessions, including abstentions and coding failures. Each arm passed 54 of 56 strict assessments; the full 128-record comparison pool passed 124. One task's ambiguous exception wording limits interpretation of its four retained failures.

The commands-first policy had a 4.84% lower observed uniform API-equivalent cost and a 1.17% higher observed workflow time than pass-through. Both nominal 95% paired intervals include zero. The study therefore does not establish

simultaneous cost and speed superiority, a general cost saving, or quality equivalence. Shorter observed native execution was offset by preparation and verification work in the measured workflow.

The contribution is an auditable implementation and an explicit measurement of its resource tradeoff with uncertainty. Subsequent work should use independently reviewed task contracts, a prospectively fixed panel and additional models, and should compare specific preparation choices in randomized arms before attributing benefits to source selection, command cards or deferred verification.

9 Artifact and reproducibility

The public repository is https://github.com/iamsreecharan/Memglide. The updated release contains the manuscript and supplement, editable section sources, bibliography, figure sources, the evaluated Memglide implementation, and a portable numeric export in research/results/router-v7-20261021/. The export includes all 112 primary session measurements, the full 128-session comparison pool, task-paired values, public task descriptions and provenance, protocol information, checksums and an offline statistical reproduction script. It distinguishes original evidence commitments from hashes of the portable export. The original final assessment has SHA-256 fcd5ded7f3c73799cad03a1fda0554294bc974ba34625b6680ee323ef5c94b26.

The numeric export permits reproduction of the reported arithmetic; it is not an independent replication or a complete execution image. Authentication, installed dependency trees and platform state are excluded. Private grading material and complete native transcripts from the new comparison are not part of this public numeric export. Source commitments identify the preserved records without claiming that a checksum alone makes unpublished material publicly auditable. The complete download retains the earlier v6 evidence capsule in a clearly marked historical directory, separate from the new data. That capsule also remains available at its immutable historical GitHub revision.

The supplement describes cohort accounting, amendments, timing, grading and reproduction boundaries. Recomputing tables and rebuilding the PDFs make no model calls. Repeating native collection requires separately authenticated Codex access, the declared runtime and a new allocation; the sealed allocation is not reopened. No submission, acceptance, peer review, archive DOI or independent human replication is claimed.

10 AI assistance

OpenAI Codex assistants supported implementation, task and control generation, experiment orchestration, analysis code, literature-source checks and manuscript drafting. Separate agents reviewed task contracts and experimental records. These automated reviews do not constitute independent human replication. The coding model used as an experimental subject is specified in the methods. No AI tool is listed as an author. Verification of the final manuscript and responsibility for any submission rest with the named author, Sree Charan Reddy Kailasam.

References

[1] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 13358–13376. doi:10.18653/v1/2023.emnlp-main.825

[2] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66

[3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638

[4] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *International Conference on Learning Representations*. https://arxiv.org/abs/2306.03091

[5] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489* (2024). https://arxiv.org/abs/2407.01489

[6] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. https://arxiv.org/abs/2405.15793

[7] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2471–2484. doi:10.18653/v1/2023.emnlp-main.151